Module 16 — Governance, Compliance and Responsible AI

> **Goal:** Turn governance from paperwork into engineering: risk-tier use cases, map regulatory and standards obligations to controls and evidence, generate documentation from registries, keep tamper-evident audit trails, measure fairness and oversight quality, and manage licence and vendor risk. Regulatory content is orientation only, not legal advice; verify current text with counsel.

Table of Contents

- 16.1 Governance Operating Model: Risk Tiering, Roles and Review Gates
- 16.2 Regulatory and Standards Landscape: EU AI Act, NIST AI RMF, ISO/IEC 42001
- 16.3 Documentation Artefacts: Model Cards, System Cards, Datasheets and the AI-BOM
- 16.4 Audit Trails, Retention and Privacy by Design
- 16.5 Fairness, Transparency and Human Oversight
- 16.6 Licences, Intellectual Property and Third-Party Risk

16.1 Governance Operating Model: Risk Tiering, Roles and Review Gates

Conceptual Deep Dive

Governance works when it follows a hierarchy that ends in machine-checkable evidence: **principles** (what we stand for) lead to **policies** (what must hold), **standards** (how, per technology), **controls** (specific mechanisms) and finally **evidence** (artefacts proving the control ran). If the chain stops at a PDF, you have paperwork. The practical test is whether CI, the model registry and tracing emit the evidence without a human filling in a form.

The core mechanism is **use-case intake and risk tiering**. Score each proposal on impact on people, autonomy (suggests, acts with approval, acts alone), data sensitivity, scale, reversibility and user vulnerability. The **risk tier** then selects required controls: eval depth, human oversight, red-teaming, documentation and monitoring. A low tier (internal summariser) needs self-attestation and an inventory entry; a high tier (credit or hiring decisions) needs a review board, a system card, independent red-teaming and periodic re-certification.

Roles matter more than committees: a product owner accountable for outcomes, an engineering owner for controls, security and privacy for their domains, legal for obligations, a domain SME for ground truth, and a model-risk reviewer who is independent of the builders for high tiers.

Lifecycle gates: design review (tier assigned), pre-launch readiness (the manifest linter from Module 12 section 3.3 checks the bundle is complete and gated), post-launch monitoring review after the first weeks, and periodic re-certification. An **AI register** lists every model, prompt bundle, dataset, vendor and use case with owner and tier; it is the denominator for all governance metrics.

Exceptions are normal but must expire and have a named approver. Track coverage of the inventory, time-to-approval, overdue reviews and exception count; automate evidence collection from CI, registries and trace stores so reviewers read results rather than ask for them.

Edge cases. **Shadow AI** (teams using unapproved tools) is found through egress logs, expense and procurement data, and browser-extension audits, then handled by offering an approved alternative rather than

only banning. **Scope creep** silently promotes a low-tier tool: define re-tier triggers (new data class, new user population, tool or write access added, autonomy raised, volume past a threshold) and wire them to the change workflow.

Cross-Connection Scenarios

- The release manifest linter (Module 12 section 3.3) is the technical enforcement point for the pre-launch gate.
- Tier-dependent eval depth connects to the release gates in Module 13 and the golden sets in Module 07.
- Evidence collection reads from tracing (Module 10) and the registry (Module 01).
- Tier-dependent guardrail fail-open versus fail-closed decisions are covered in 17.5.

Production Anti-Patterns

- Governance theatre: long forms with no technical enforcement, so teams learn to fill forms and ignore controls.
- One-size-fits-all review that queues a weekend prototype behind a hiring-model review, which drives teams to shadow AI.
- No inventory, so after an upstream licence or vulnerability notice nobody can say what is deployed.

Minimal Working Code

A risk-tier classifier turns a questionnaire into a tier and a cumulative list of required controls.

```
CONTROLS = {
    1: ["inventory entry", "self-attestation"],
    2: ["basic eval set", "tracing", "owner sign-off"],
    3: ["red-team", "human oversight", "system card", "monitoring revi
ew"],
    4: ["review board", "independent validation", "periodic re-certification&quo
t;],
}

def classify(q: dict) -> tuple[int, list[str]]:
    """q values 0-3: impact, autonomy, data, scale, reversibility; vulnerable_users: bool.
"""
    score = sum(q[k] for k in ("impact", "autonomy", "data", "scale&qu
ot;, "reversibility"))
    tier = 1 if score <= 3 else 2 if score <= 7 else 3 if score <= 11 else 4
    if q.get("vulnerable_users") or q["impact"] == 3:
        tier = max(tier, 3)          # hard floor: people-affecting decisions
    required = [c for t in range(1, tier + 1) for c in CONTROLS[t]]
    return tier, required

code_assistant = dict(impact=1, autonomy=1, data=2, scale=1, reversibility=0)
loan_bot = dict(impact=3, autonomy=1, data=3, scale=2, reversibility=2, vulnerable_users=True)
print(classify(code_assistant)[0], classify(loan_bot)[0])
assert classify(loan_bot)[0] > classify(code_assistant)[0]
```

An AI register checks evidence completeness per tier and flags overdue reviews against a supplied date.

```
from datetime import date, timedelta

EVIDENCE = {1: {"owner"}, 2: {"owner", "eval_report"}, 3: {"owner"
;, "eval_report", "system_card", "redteam"}}
REVIEW_DAYS = {1: 365, 2: 180, 3: 90}

def audit(register: list[dict], today: date) -> list[str]:
    issues = []
    for s in register:
        missing = EVIDENCE[s["tier"]] - set(s["evidence"])
```

```
        if missing:
            issues.append(f"{s['name']}: missing {sorted(missing)}")
        due = s["last_review"] + timedelta(days=REVIEW_DAYS[s["tier"]])
        if due < today:
            issues.append(f"{s['name']}: review overdue by {(today - due).days} days")
    return issues

reg = [
    {"name": "code-assist", "tier": 2, "evidence": ["owner&q
uot;, "eval_report"], "last_review": date(2026, 3, 1)},
    {"name": "loan-faq", "tier": 3, "evidence": ["owner"
;, "eval_report"], "last_review": date(2026, 5, 1)},
]
for line in audit(reg, date(2026, 9, 1)):
    print(line)
```

Interview Angle

Q: How do you introduce AI governance in a company that ships fast without slowing teams? **A:** Tier by risk so 80 percent of use cases need only registration and self-attestation, and put controls in CI and the registry (paved road) so compliance is a by-product. Trade-off: automation costs platform effort up front and encodes policy in code that needs its own owner. Measure median time-to-approval per tier, inventory coverage (registered versus discovered via egress logs) and overdue reviews; if low-tier approval exceeds a day, the process is too heavy.

Q: How do you decide the tier of an internal code assistant versus a customer-facing loan FAQ bot? **A:** Score impact, autonomy, data, scale, reversibility and vulnerability. The code assistant touches proprietary code but a human reviews every change, so it is mid-tier with secrets and licence controls. The loan bot influences financial decisions for the public, so it sits higher: grounded answers, disclaimers, escalation, red-teaming. Trade-off: scoring is subjective; calibrate by back-testing past cases and checking that two reviewers agree, for example above 80 percent.

Q: What evidence would you automate collecting? **A:** Eval reports with intervals, bundle and model versions, red-team results, guardrail configuration, approval records, trace samples and monitoring dashboards, all linked from the registry entry. Trade-off: more evidence means more storage and sensitive-data exposure, so keep references and hashes. Check: an auditor can reproduce one launch decision from the register within an hour.

16.2 Regulatory and Standards Landscape: EU AI Act, NIST AI RMF, ISO/IEC 42001

Conceptual Deep Dive

This is orientation, not legal advice. Dates, thresholds and obligations change, so treat every specific below as **verify current text** and involve counsel before relying on it.

The **EU AI Act** is risk-based: prohibited practices; high-risk systems (use cases listed in annexes, such as employment, education, credit and essential services); transparency obligations for certain systems (for example disclosing that users interact with AI, and labelling synthetic content); and minimal risk for the rest. It separately regulates providers of general-purpose AI (GPAI) models, with extra duties where a model is judged to carry systemic risk. Roles matter: a **provider** places a system on the market, a **deployer** uses it, and obligations differ. Applicability is phased over several years (verify the schedule). Typical high-risk requirements: a risk-management system, data governance, technical documentation, record-keeping and logging, human oversight, and accuracy, robustness and cybersecurity.

The **NIST AI RMF 1.0** is voluntary and organised around four functions: GOVERN, MAP, MEASURE, MANAGE. Its Generative AI Profile adds generative-specific risks and suggested actions and works well as a checklist source. **ISO/IEC 42001** is a certifiable AI management-system standard (plan-do-check-act for the organisation), with **ISO/IEC 23894** giving risk-management guidance. Management-system certification says your organisation runs a process; it is not conformity of a particular product, which is what product regulation asks for.

Adjacent regimes often bind first: GDPR (lawful basis, DPIA for high-risk processing, rights around solely automated decisions), sector rules (financial model-risk management, health privacy), and national or US state laws on automated decisions, deepfakes or disclosure.

The engineering move is a **mapping table**: obligation, control, evidence, owner. One control (for example logging with 16.4) satisfies many obligations, so build controls once and cite them many times. Run horizon scanning on a cadence (quarterly) with a named owner, and record "last verified" dates on every regulatory row.

Edge cases. Using a hosted GPAI model versus fine-tuning or substantially modifying one may change your role and obligations; ask counsel before assuming you are only a deployer. For multi-jurisdiction products, apply the strictest common control where it is cheap (disclosure, logging, oversight) and gate jurisdiction-specific features by region only where the cost is high.

Source	Nature	Certifiable	Use it for
EU AI Act	Law	Conformity assessment (high-risk)	Legal obligations
NIST AI RMF	Voluntary framework	No	Risk process and checklists
ISO/IEC 42001	Management standard	Yes	Organisation-level assurance

Cross-Connection Scenarios

- Tiering (16.1) is how you decide which systems get the high-risk control set.
- Logging and human oversight obligations are implemented by 16.4 and 16.5.
- Technical documentation maps to cards and the AI-BOM in 16.3.
- Security controls in Module 08 supply the cybersecurity and robustness evidence.

Production Anti-Patterns

- Treating one framework as compliance with all others, then discovering the missing obligations at audit.
- Quoting regulatory dates and thresholds from memory in design docs, which go stale silently.
- Compliance by policy document with no technical evidence, which fails the first request to demonstrate a control.

Minimal Working Code

An obligations-to-controls table as data, plus a gap analyser against a system's registry entry.

```
OBLIGATIONS = [  # illustrative rows; verify against current text
    {"obligation": "record-keeping", "control": "audit_log", &quo
t;evidence": "log schema + sample", "owner": "platform"},
    {"obligation": "human oversight", "control": "approval_queue"
, "evidence": "override logs", "owner": "product"},
    {"obligation": "technical documentation", "control": "system_card&
quot;, "evidence": "card in bundle", "owner": "eng"},
    {"obligation": "risk management", "control": "risk_register",
 "evidence": "review minutes", "owner": "risk"},
    {"obligation": "robustness", "control": "redteam_suite", &quo
t;evidence": "suite report", "owner": "security"},
]

def gaps(system: dict, applicable: set[str]) -> list[dict]:
    present = set(system["controls"])
    return [o for o in OBLIGATIONS if o["obligation"] in applicable and o["control"]
not in present]

sys_entry = {"name": "hr-screener", "controls": ["audit_log", &qu
ot;system_card"]}
for g in gaps(sys_entry, {"record-keeping", "human oversight", "risk management&
quot;}):
    print(f"GAP {g['obligation']}: add {g['control']} (owner {g['owner']})")
```

An illustrative category classifier, encoded with tests.

```
def ai_act_style_category(a: dict) -> str:
    """Illustrative decision tree. NOT legal advice; categories and annex lists must be ve
rified."""
    if a.get("social_scoring") or a.get("manipulative_techniques"):
        return "prohibited"
```

```
    high_domains = {"employment", "education", "credit", "essential_se
rvices", "law_enforcement"}
    if a.get("domain") in high_domains and a.get("influences_decisions", True):
        return "high-risk"
    if a.get("interacts_with_humans") or a.get("generates_synthetic_content"):
        return "transparency"
    return "minimal"

assert ai_act_style_category({"domain": "employment"}) == "high-risk"
assert ai_act_style_category({"domain": "employment", "influences_decisions"
;: False, "interacts_with_humans": True}) == "transparency"
assert ai_act_style_category({"social_scoring": True}) == "prohibited"
assert ai_act_style_category({"domain": "internal_search"}) == "minimal"
print("classifier tests passed")
```

Interview Angle

Q: What would you do in the first month to prepare an LLM product for the EU AI Act? **A:** Build the register, classify each system with counsel (provider or deployer, category), then map obligations to existing controls and list gaps with owners. Typical gaps: logging, documentation, oversight evidence. Trade-off: preparing for the strictest reading costs time; sequence by tier and launch date. Check: every high-risk system has a gap list with owners and dates, and every mapping row carries a "last verified" date.

Q: How do NIST AI RMF and ISO 42001 differ, and how do they complement each other? **A:** NIST AI RMF is a voluntary risk framework and vocabulary; ISO/IEC 42001 is a certifiable management-system standard. Use RMF to structure risk work and 42001 to give it an auditable organisational wrapper. Trade-off: certification adds cost and audit overhead that only pays off when customers or regulators ask. Check: a risk found by MAP/MEASURE flows into a tracked corrective action in the management system.

Q: Our assistant helps screen job applicants. What changes? **A:** Employment is a commonly high-risk domain, so expect the heavy control set: risk management, data governance, logging, human oversight with real authority, bias testing, candidate notices and a way to contest outcomes, plus GDPR automated-decision and DPIA questions. Trade-off: keeping the tool to ranking support lowers autonomy but not necessarily the category. Check: a bias test by slice and a reviewer override rate, both reviewed by counsel.

16.3 Documentation Artefacts: Model Cards, System Cards, Datasheets and the AI-BOM

Conceptual Deep Dive

Four artefacts cover different questions. A **model card** (Mitchell et al., 2019) documents a model: intended and out-of-scope use, training data summary, evaluation by slice, limitations and ethical considerations. A **datasheet** for datasets (Gebru et al.) documents motivation, composition, collection process, preprocessing, uses and maintenance. A **system card** describes the end-to-end system that users actually touch: the model or models, prompts, retrieval sources, tools, guardrails, human oversight, monitoring and known failure modes. The **AI-BOM** (AI bill of materials) lists components with versions, hashes and licences: models, adapters, datasets, prompt bundles, indexes, embedding models and libraries. Formats exist (CycloneDX with ML-BOM support and SPDX with an AI profile); verify current versions.

The key design rule is a single source of truth. Generate cards from the registry and eval outputs rather than writing them by hand, and version them inside the release bundle (13.1), so a document and the thing it describes cannot drift. A hand-written card is wrong by the second release.

Contents checklist: intended uses and explicit out-of-scope uses; known failure modes; evaluation results with confidence intervals and per-slice breakdowns; safety and red-team testing summary; data and training summary; licences; monitoring plan; escalation contact; last-updated date and bundle id. Every claim should link to evidence (an eval run id, a trace query).

Audiences differ. Users need plain-language scope and limits; auditors need traceable evidence; engineers need failure modes and reproduction details. Produce layered views from one source rather than three documents. Security-sensitive content (prompt text, detector thresholds, red-team attack recipes) goes in the internal version only; the public version states capabilities and limits.

Staleness checks belong in CI: fail the build if the card is older than N days, if the bundle hash changed without a card regeneration, or if required fields are empty. Treat unfilled "TBD" the same as missing.

Edge cases. For a closed vendor model you cannot inspect, document what you can verify: your own evals on your slices, the pinned model identifier and date, and vendor documentation cited by link and retrieval date; state plainly what is unknown (training data, architecture). When a vendor silently updates a model alias, your card is wrong until re-evaluated, so pin dated versions where offered.

Cross-Connection Scenarios

- Release bundles and manifests (13.1) carry the generated documents.
- Slice-level results come from the evaluation harnesses in Module 07 and fairness work in 16.5.
- Licence fields in the AI-BOM feed the checker in 16.6.
- Regulatory technical-documentation obligations (16.2) map to the system card.

Production Anti-Patterns

- Hand-written cards updated once at launch, then quoted to auditors a year later.
- Claims such as "robust to jailbreaks" with no evidence link, which cannot be defended when challenged.
- Reporting averages only, hiding a failing language or customer segment.

Minimal Working Code

A system-card generator validates required fields before it renders Markdown from a registry record.

```
REQUIRED = ["name", "version", "intended_use&
quot;, "out_of_scope", "eval", "failure_modes", "owner"]

def render_card(rec: dict) -> str:
    missing = [k for k in REQUIRED if not rec.get(k)]
    if missing:
        raise ValueError(f"registry record incomplete: {missing}")
    rows = "\n".join(f"| {s} | {m['score']:.2f} | {m['ci'][0]:.2f}-{m['ci'][1]:.2f} |"
; for s, m in rec["eval"].items())
    return (f"# System card: {rec['name']} v{rec['version']}\n\n"
            f"**Intended use:** {rec['intended_use']}\n\n**Out of scope:** {rec['out_of_scope']}\n\n
"
            f"| Slice | Score | 95% CI |\n|---|---|---|\n{rows}\n\n"
            f"**Known failure modes:** {', '.join(rec['failure_modes'])}\n\n**Owner:** {rec['owner']
}\n")

rec = {"name": "support-chat", "version": "1.4", "intended_u
se": "answer product questions",
       "out_of_scope": "legal or medical advice", "owner": "support-pl
atform",
       "eval": {"overall": {"score": 0.91, "ci": (0.89, 0.93)}, &
quot;hindi": {"score": 0.82, "ci": (0.76, 0.87)}},
       "failure_modes": ["stale policy docs", "long multi-turn drift"]}
print(render_card(rec))
try:
    render_card({"name": "x"})
except ValueError as e:
    print("blocked:", e)
```

An AI-BOM emitter hashes components and diffs two release BOMs.

```
import hashlib, json

def bom(components: dict[str, bytes], licences: dict[str, str]) -> dict:
    return {n: {"sha256": hashlib.sha256(b).hexdigest()[:16], "licence": licences.get
(n, "UNKNOWN")}
            for n, b in sorted(components.items())}

def diff(old: dict, new: dict) -> dict:
    return {"added": sorted(set(new) - set(old)), "removed": sorted(set(old) - set(ne
w)),
            "changed": sorted(n for n in set(old) & set(new) if old[n] != new[n])}
```

```
v1 = bom({"model": b"weights-v1", "prompt": b"You are helpful", &
quot;index": b"idx-1"},
        {"model": "apache-2.0", "prompt": "internal", "inde
x": "internal"})
v2 = bom({"model": b"weights-v1", "prompt": b"You are concise", &
quot;index": b"idx-1", "reranker": b"rr"},
        {"model": "apache-2.0", "prompt": "internal", "inde
x": "internal", "reranker": "mit"})
print(json.dumps(diff(v1, v2)))
assert diff(v1, v2)["changed"] == ["prompt"]
```

Interview Angle

Q: What belongs in a system card that is not in a model card? **A:** The deployed context: prompts, retrieval sources, tools and permissions, guardrails, human-oversight design, monitoring and escalation, plus system-level eval (task success, safety, per-slice). A model card describes weights; harms often come from the combination. Trade-off: system cards expose attack surface, so split internal and public versions. Check: every claim links to an eval run on the current bundle id.

Q: How do you keep documentation true as the system changes weekly? **A:** Generate it from the registry and eval outputs in CI, version it with the release bundle, and fail the build on missing fields or a card older than the bundle. Trade-off: generation rigidity versus narrative richness; keep a short hand-written section for context that is reviewed on tier-based cadence. Check: zero releases where card bundle id differs from deployed bundle id.

Q: How would you document a system built on a vendor model you cannot inspect? **A:** Record pinned model identifier and retrieval date, cite vendor documentation, and publish your own evals on your slices as the evidence, with explicit "unknown" entries for training data. Trade-off: you cannot certify properties you did not test. Check: a scheduled re-eval detects vendor-side drift; alert when scores move beyond the noise floor.

16.4 Audit Trails, Retention and Privacy by Design

Conceptual Deep Dive

An **audit trail** exists to reconstruct a decision and assign accountability. Record who (user, service identity, approver), what (action, inputs by reference, output), when, which versions (bundle, model alias resolved to a dated version, prompt version, index snapshot), which retrieved document ids, which tool calls with arguments hashes, which approvals or overrides, and the outcome. Avoid raw personal data: store references, hashes or redacted content under policy, and keep full text only in tiers where reconstruction demands it, under tighter access.

Tamper evidence has layers. A **hash-chained** record set (each entry includes the hash of the previous one) makes silent edits and deletions detectable on verification; append-only or WORM (write-once-read-many) storage prevents them; periodic anchoring (publishing the latest chain head to a separate system or custodian) protects against rewriting the whole chain. The operators of the system must not hold write or delete rights on the audit store; access to the store is itself logged.

Retention is a schedule by record class (operational traces, audit records, prompts, user content), with a legal-hold override and verified deletion (prove the purge ran). It must coordinate with erasure obligations in 8.7. Privacy by design adds: a documented lawful basis and purpose, data minimisation, a **DPIA** where processing is likely high-risk (typically profiling, sensitive data, large-scale monitoring, vulnerable groups; check with the privacy team), transparency notices, and a route for users to contest automated outcomes with human review. Cross-border transfers, data residency and vendor subprocessors need a register and contract terms.

Auditors want evidence, not exports: package reproducible queries over the audit store ("all refunds above a threshold approved without review in Q2") with stored results and hashes of those results, so the answer can be regenerated.

Edge cases. Audit retention can conflict with a deletion request: define precedence with counsel in advance (commonly: delete content, retain minimal pseudonymised evidence of the decision where a legal duty requires it). Logging every full trace at volume is costly and risky: sample low-risk traffic, capture high-risk fully, and always capture metadata.

A tested restore matters as much as the log: an audit store that cannot be restored, or whose chain breaks after a migration, fails the first investigation.

Cross-Connection Scenarios

- Tracing infrastructure and PII scrubbing (Module 10) are the capture layer; the audit store is a separate, stricter sink.
- Erasure and privacy engineering (8.7) define deletion mechanics that retention must respect.
- Agent approvals and idempotency (Module 09, 18.4) are the events an audit trail must record.
- Incident reconstruction (17.6) depends on the same records.

Production Anti-Patterns

- Audit logs editable by the engineers who operate the system, so they prove nothing.
- Logging full prompts "just in case", creating a personal-data store nobody budgeted for or secured.
- No tested restore of audit data, discovered missing during the first regulator request.

Minimal Working Code

A hash-chained audit log with append, verify and a tamper test.

```
import hashlib, json

class AuditLog:
    def __init__(self):
        self.entries = []

    def append(self, event: dict) -> str:
        prev = self.entries[-1]["hash"] if self.entries else "0" * 64
        body = json.dumps(event, sort_keys=True)
        h = hashlib.sha256((prev + body).encode()).hexdigest()
        self.entries.append({"event": event, "prev": prev, "hash": h})
        return h

    def verify(self) -> int | None:
        prev = "0" * 64
        for i, e in enumerate(self.entries):
            body = json.dumps(e["event"], sort_keys=True)
            if e["prev"] != prev or hashlib.sha256((prev + body).encode()).hexdigest() != e[&qu
ot;hash"]:
                return i
            prev = e["hash"]
        return None

log = AuditLog()
for amt in (20, 45, 300):
    log.append({"actor": "agent-7", "action": "refund", "amo
unt": amt, "approved_by": "u12"})
assert log.verify() is None
log.entries[1]["event"]["amount"] = 4
print("tampered entry index:", log.verify())
assert log.verify() == 1
```

A retention enforcer produces a dry-run purge report that respects legal holds.

```
from datetime import date, timedelta

POLICY_DAYS = {"trace": 30, "prompt_text": 14, "audit": 2555}

def purge_plan(records: list[dict], holds: set[str], today: date) -> dict:
    plan = {"purge": [], "kept_hold": [], "kept": []}
    for r in records:
        expired = r["created"] + timedelta(days=POLICY_DAYS[r["cls"]]) < today
        if not expired:
            plan["kept"].append(r["id"])
        elif r["subject"] in holds:
            plan["kept_hold"].append(r["id"])
        else:
            plan["purge"].append(r["id"])
    return plan
```

```
recs = [
    {"id": "t1", "cls": "trace", "created": date(2026,
1, 1), "subject": "a"},
    {"id": "t2", "cls": "trace", "created": date(2026,
1, 1), "subject": "legal-case-9"},
    {"id": "p1", "cls": "prompt_text", "created": date(
2026, 8, 25), "subject": "b"},
    {"id": "a1", "cls": "audit", "created": date(2026,
1, 1), "subject": "a"},
]
print(purge_plan(recs, {"legal-case-9"}, date(2026, 9, 1)))
```

Interview Angle

Q: Design the audit trail for an agent that approves refunds. **A:** Append-only, hash-chained records per decision: request id, customer reference (not raw PII), policy version, bundle and model version, retrieved ids, tool calls with argument hashes, approver identity and timestamp, outcome and amount. Separate write path from operator rights. Trade-off: richer records raise privacy and storage cost; keep content by reference. Check: a nightly job verifies the chain and a quarterly restore test reconstructs a random decision.

Q: A regulator asks you to reconstruct one decision from six months ago. What must exist? **A:** The trace id, inputs by reference, exact bundle, model version and prompt version, retrieved document ids with the index snapshot, tool results, approvals and outcome, retained past the period. Trade-off: snapshots of indexes cost storage; keep content hashes plus versioned sources. Check: run the reconstruction drill on a random six-month-old decision and compare the replayed output class with the logged outcome.

Q: How do you reconcile audit retention with a deletion request? **A:** Agree precedence with counsel in advance. A common design deletes content and identifiers while retaining a minimal pseudonymised decision record where a legal duty applies, and tells the user what is kept and why. Trade-off: partial deletion may not satisfy every regime. Check: a deletion test proves the content is gone from every store, including backups on their expiry cycle, while chain verification still passes.

16.5 Fairness, Transparency and Human Oversight

Conceptual Deep Dive

Start with a **harm taxonomy** for your use case: allocative (who gets a job, loan, service), quality-of-service (worse performance for a dialect or language), representational (stereotypes in output) and safety harms. Name the affected groups and the decision they influence before choosing metrics.

Measurement uses several tools. **Slice performance parity** compares accuracy or task success across groups with confidence intervals. **Counterfactual prompt pairs** hold the request fixed while swapping names, dialect, gender terms or other attributes and measure output differences. Also compare refusal rates and tone, and toxicity by group, remembering that automated toxicity classifiers and LLM judges carry their own group biases. Ratios such as the four-fifths rule come from US employment practice and are context specific; use them as screening heuristics, not universal thresholds.

Mitigations sit at several layers: data and prompt changes, retrieval fixes (biased sources), threshold changes, constraining the model to structured criteria, and human review. Track **residual risk** after mitigation and re-run the probes on every release, because prompt edits shift behaviour.

Transparency is practical: disclose that users are interacting with AI, show citations and uncertainty cues, state limits, and give a contest route. Accessibility and multilingual equity are quality dimensions: measure task success per language and with assistive technology, not only English desktop.

Human oversight is meaningful only if reviewers have authority to override, context to judge, time to read, and training; otherwise it is a rubber stamp. Automation bias (accepting model suggestions) is measured with seeded-error audits: inject known-bad items into the queue and compute the miss rate. Log overrides and examine them; a zero override rate is a warning sign, not a success. Size review queues from capacity: if flagged volume exceeds reviewer throughput, attention degrades and backlog hides risk; route by risk score and sample the rest.

Edge cases. Small groups give wide intervals; pool or collect more data and do not claim parity from n=12. Fairness definitions conflict (equal opportunity versus calibration versus demographic parity); you generally cannot satisfy all, so document the chosen definition, why, and who approved.

Cross-Connection Scenarios

- Slice reporting feeds the cards in 16.3; the harm taxonomy feeds tiering in 16.1.
- Judge bias and calibration (14.2) apply to any LLM-based toxicity or quality scoring.
- Red-teaming for harmful content (Module 08) overlaps with representational-harm probing.
- Oversight design connects to approval workflows in Module 09 and 18.4.

Production Anti-Patterns

- Routing 100 percent of outputs to reviewers who approve in two seconds each, creating liability without protection.
- Declaring fairness from an aggregate metric while a language or customer segment fails.
- Hiding AI involvement from users, which removes consent and the ability to contest.

Minimal Working Code

A counterfactual probe on a fake biased model, with a permutation test for the disparity.

```
import hashlib, random

def fake_model(prompt: str) -> int:
    """1 = recommend interview. Deliberately biased against one name for the demo."&q
uot;"
    u = int(hashlib.md5(prompt.encode()).hexdigest(), 16) % 100 / 100   # deterministic noise
    return 0 if "Jamal" in prompt and u < 0.35 else 1

TEMPLATE = "Candidate {name} has 5 years of Python experience (case {i}). Recommend interview?"
def outcomes(name: str, n: int = 200) -> list[int]:
    return [fake_model(TEMPLATE.format(name=name, i=i)) for i in range(n)]

def permutation_p(a: list[int], b: list[int], iters: int = 2000, seed: int = 0) -> float:
    rng = random.Random(seed)
    obs = abs(sum(a) / len(a) - sum(b) / len(b))
    pool, hits = a + b, 0
    for _ in range(iters):
        rng.shuffle(pool)
        x, y = pool[:len(a)], pool[len(a):]
        hits += abs(sum(x) / len(x) - sum(y) / len(y)) >= obs
    return (hits + 1) / (iters + 1)

a, b = outcomes("Emily"), outcomes("Jamal")
print(sum(a) / len(a), sum(b) / len(b), "p =", round(permutation_p(a, b), 4))
```

A review-queue planner with seeded known-bad items estimates the reviewer miss rate.

```
import random

def plan(flagged_per_day: int, reviewers: int, items_per_reviewer: int) -> dict:
    cap = reviewers * items_per_reviewer
    return {"capacity": cap, "backlog_growth": max(0, flagged_per_day - cap), "u
tilisation": flagged_per_day / cap}

def seeded_audit(review_fn, n_clean=180, n_bad=20, seed=1) -> float:
    rng = random.Random(seed)
    items = [("clean", i) for i in range(n_clean)] + [("bad", i) for i in range(n_bad
)]
    rng.shuffle(items)
    missed = sum(1 for kind, _ in items if kind == "bad" and review_fn(kind, rng) == "appr
ove")
    return missed / n_bad

tired = lambda kind, rng: "approve" if rng.random() < 0.40 else "reject"   # fake
reviewer behaviour
print(plan(600, 4, 120))
print("estimated miss rate:", seeded_audit(tired))
```

Interview Angle

Q: How would you test an HR assistant for bias before launch? **A:** Define the decision it influences, build counterfactual prompt pairs (names, gender terms, dialect) and slice evaluations on labelled cases, compare score and refusal differences with intervals and a permutation test, and review with HR and counsel. Trade-off: probes catch only the attributes you vary; add monitoring for proxies. Check: disparity within an agreed bound with the interval stated, re-run on every prompt or model change.

Q: How do you design human-in-the-loop so it is not just a rubber stamp? **A:** Give reviewers authority, context (sources, confidence) and time; route only risky cases; seed known-bad items and track the miss rate; log overrides and review them. Trade-off: more review costs money and latency; reduce volume with better triage. Check: seeded-error miss rate below an agreed target, and override rate that is neither zero nor implausibly high.

Q: What do you tell users about AI involvement and its limits? **A:** That they are interacting with AI, what it can and cannot do, when a human takes over, how to contest an outcome, and where its answers come from via citations. Trade-off: heavy disclaimers reduce usability and get ignored; use short, placed notices. Check: user testing shows people can state the limit correctly, and contest requests are resolved within the stated time.

16.6 Licences, Intellectual Property and Third-Party Risk

Conceptual Deep Dive

Start with **model licences**. Some are permissive (Apache-2.0, MIT style); others are custom community licences with acceptable-use restrictions, naming or attribution requirements, user-count thresholds or restrictions on using outputs to train other models. Read the actual text, not the hub label, and keep a **licence inventory** per model, adapter and dataset, tied to the AI-BOM (16.3). Copy and archive the licence text at adoption time.

Data raises separate questions: dataset licences, scraped-content risk, personal data in the corpus, and copyright positions that vary by jurisdiction and are still developing (take counsel's advice rather than rules of thumb). For RAG, you also need the right to index and display customer or third-party documents. Output ownership terms differ by vendor. Code assistants add contamination risk: generated code resembling licensed code; mitigate with licence scanners and policy on acceptance. Distillation adds teacher-model terms: some providers restrict using outputs to train competing models (see 15.5), and synthetic data inherits provenance constraints from its generator.

Vendor due diligence for LLM APIs: security attestations (SOC 2 reports, ISO 27001 certificates, scope and dates), data-processing agreement, subprocessors, data retention and training-on-your-data terms, residency options, incident-notification times, indemnities, rate-limit and deprecation policies, and an **exit plan** (alias abstraction, tested fallback model, exportable data). Assess criticality: a vendor on the critical path of a high-tier system deserves deeper review than a summariser used by one team.

Supply chain: dependency scanning, pinned versions and hashes, model provenance (publisher identity, signatures where available), and safetensors-only loading to avoid pickle code execution (Module 08 section 3.12). Maintain a **vendor risk register** with owner, criticality, last review and open findings, reviewed on a cadence.

Edge cases. An upstream licence can change after adoption: pin the version, keep a copy of the licence text that applied, and review before upgrading. A merged model (Module 04 merging) inherits the most restrictive terms of its parents, and incompatible licences may make merging impermissible; check each parent and the derivative terms.

Check	Question	Evidence
Licence	Commercial use, thresholds, output restrictions?	Archived text, inventory row
Data	Right to train/index/display?	Dataset licence, DPA
Vendor	Retention, training use, residency, exit?	DPA, attestation, tested fallback

Cross-Connection Scenarios

- AI-BOM licence fields (16.3) are the input to the compatibility checker below.
- Model merging and adapters (Module 04) create mixed-licence artefacts.
- Safe model loading and supply-chain security live in Module 08.
- Fallback and exit plans connect to gateway aliasing (17.3) and failover (17.5).

Production Anti-Patterns

- Adopting a model because it trends on a hub without reading its licence, then finding a restriction at contract time.
- Single-vendor dependency on a critical path with no tested exit plan.
- Assuming synthetic data has no provenance constraints, ignoring the generator's terms.

Minimal Working Code

An illustrative licence-compatibility check (not legal advice) compares inventory entries to restriction flags for a given use.

```
RESTRICTIONS = {  # illustrative table, verify real licence text
    "apache-2.0": set(),
    "mit": set(),
    "community-custom": {"user_threshold", "no_train_other_models"},
    "non-commercial": {"commercial"},
    "UNKNOWN": {"unreviewed"},
}

def check(inventory: list[dict], use: set[str]) -> list[str]:
    problems = []
    for c in inventory:
        hit = RESTRICTIONS.get(c["licence"], {"unreviewed"}) & use
        if hit:
            problems.append(f"{c['name']} ({c['licence']}): blocked by {sorted(hit)}")
    return problems

inv = [{"name": "base-model", "licence": "community-custom"},
       {"name": "embedder", "licence": "apache-2.0"},
       {"name": "eval-data", "licence": "non-commercial"}]
for p in check(inv, {"commercial", "user_threshold", "no_train_other_models"
;}):
    print(p)
```

A vendor-risk scorecard with weighted criteria and must-have gates.

```
WEIGHTS = {"security": 0.3, "privacy": 0.3, "
reliability": 0.2, "exit": 0.2}
GATES = {"no_training_on_data": True, "dpa_signed": True}

def score(vendor: dict) -> tuple[float, list[str]]:
    failed = [g for g, need in GATES.items() if vendor["gates"].get(g) != need]
    total = sum(WEIGHTS[k] * vendor["scores"][k] for k in WEIGHTS)   # scores 0-5
    return round(total, 2), failed

a = {"scores": {"security": 4, "privacy": 4, "reliability": 3, &q
uot;exit": 2}, "gates": {"no_training_on_data": True, "dpa_signed": Tr
ue}}
b = {"scores": {"security": 5, "privacy": 5, "reliability": 5, &q
uot;exit": 4}, "gates": {"no_training_on_data": False, "dpa_signed": T
rue}}
for name, v in (("A", a), ("B", b)):
    s, failed = score(v)
    print(name, s, "REJECT" if failed else "ok", failed)
```

Interview Angle

Q: Can we fine-tune an open-weight model for a commercial product? What do you check? **A:** Read the licence text for commercial use, user thresholds, output-use and attribution terms, and whether derivatives must carry the same licence; then check dataset licences, and whether the base's terms restrict training or distilling other models. Trade-off: permissive models cost capability or size. Check: a signed-off inventory row per component with archived licence text and counsel review for custom licences.

Q: What would you require from an LLM API vendor before sending customer data? **A:** A DPA, no training on our data by default, retention limits or zero retention, subprocessor list, residency options, current security attestations, incident-notification terms and a tested exit path. Trade-off: stricter terms narrow vendor choice and cost more. Check: must-have gates pass before scoring; a fallback model run in staging to prove the exit plan is real.

Q: How do you manage dependency and model supply-chain risk? **A:** Pin versions and hashes, scan dependencies, load only safetensors or verified formats, verify publisher identity, generate an AI-BOM per release and review diffs. Trade-off: pinning delays security patches, so schedule updates. Check: time from advisory to patched release in days, and zero unpinned or pickle-loaded artefacts in CI.